Project Report

on

AI-Based Sentiment Analysis System

in partial fulfillment for the award of the degree of

MASTER OF COMPUTER APPLICATIONS (AI & ML)

Subject: Natural Language Processing

Submitted by

Rahul Parmar

25MCI10280

Chandigarh University

University Institute of Computing

October 2026

BONAFIDE CERTIFICATE

Certified that this project report "AI-Based Sentiment Analysis System" is the bonafide work of "Rahul Parmar, UID: 25MCI10280" who carried out the project work under my supervision. The implementation, evaluation results, and application described in this report correspond to the Natural Language Processing project submitted for academic assessment.

SIGNATURE

_____

Nikita Pathak

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project supervisor, Nikita Pathak, for her guidance and encouragement during the development of this project. The work brought together text preprocessing, neural network training, model evaluation and a working web interface, and her feedback helped me keep these parts focused on the problem of movie-review sentiment classification.

I am thankful to the faculty of the University Institute of Computing, Chandigarh University, for the foundation in computer science and natural language processing that supported this work. I also appreciate the open-source communities behind TensorFlow, Keras, scikit-learn and Streamlit, whose tools made it possible to build and demonstrate the complete pipeline locally.

Finally, I thank my family and friends for their support throughout the project and report preparation.

Rahul Parmar
UID: 25MCI10280

ABSTRACT

This project develops an AI-based sentiment analysis system that classifies a movie review as positive or negative. It uses the IMDb movie-review benchmark distributed through Keras, with 25,000 labeled reviews for training and a separate 25,000 for final evaluation. Reviews are decoded to text, normalized by lowercasing and removing HTML breaks and non-alphanumeric characters, converted to integer sequences with a 10,000-word Keras tokenizer, and padded or truncated to 200 tokens.

The classifier is a compact neural network with a 128-dimensional embedding layer, a 64-unit Long Short-Term Memory (LSTM) layer, dropout at 0.5, and a sigmoid output. It is trained with Adam and binary cross-entropy, with a validation split and early stopping. The saved model and tokenizer are loaded by a Streamlit interface so that user-entered text follows the same preprocessing steps as the training data.

On the held-out test set the model achieved 76.60% accuracy, 80.10% precision, 70.79% recall, and 75.16% F1-score. The interface reports its raw sigmoid score and class confidence, displays evaluation metrics, and warns when a review is close to the decision threshold. The model is intended as an educational demonstration; it can misclassify reviews and its confidence is not a calibrated guarantee.

Keywords: Natural Language Processing, sentiment analysis, IMDb, LSTM, Keras, Streamlit

TABLE OF CONTENTS

Section	Page
List of Figures, Tables and Abbreviations	4
CHAPTER 1: INTRODUCTION	5
1.1 Project Scope and Planning	5
1.2 Need and Stakeholders	5
1.3 Contemporary Issues	5
1.4 Task Identification	5
1.5 Technologies Used	5
1.6 Project Timeline	5
CHAPTER 2: LITERATURE REVIEW / BACKGROUND STUDY	6
2.1 Timeline of the Problem	6
2.2 Existing Approaches	6
2.3 Literature Summary and Bibliometric Note	6
2.4 Problem Definition	6
2.5 Goals and Objectives	6
CHAPTER 3: DESIGN FLOW / PROCESS	7
3.1 Feature Selection and Design Constraints	7
3.2 Design Flow and Methodology	7
CHAPTER 4: RESULTS ANALYSIS AND VALIDATION	8
4.1 Implementation of the Solution	8
4.2 Results and Validation	9
4.3 Application Screenshots	10
CHAPTER 5: CONCLUSION AND FUTURE WORK	11
APPLICATION CODE OF PROJECT	11
REFERENCES, APPENDIX AND USER MANUAL	11

LIST OF FIGURES AND TABLES

Figure	Title	Page
Fig. 3.1	Sentiment analysis pipeline	7
Fig. 4.1	Held-out test metrics	9
Fig. 4.2	Confusion matrix	9
Fig. 4.3-4.4	Positive and negative UI predictions	10

LIST OF TABLES

Table	Title	Page
Table 1.1	Stakeholders and needs	5
Table 1.2	Technology stack	5
Table 3.1	Design constraints	7
Table 4.1	Model and training configuration	8
Table 4.2	Held-out test performance	9
Table 4.3	Confusion matrix counts	9

LIST OF ABBREVIATIONS

Abbreviation	Definition
AI	Artificial Intelligence
NLP	Natural Language Processing
IMDb	Internet Movie Database
LSTM	Long Short-Term Memory
RNN	Recurrent Neural Network
OOV	Out of Vocabulary
HTML	HyperText Markup Language
UI	User Interface
F1	Harmonic mean of precision and recall

CHAPTER 1.
INTRODUCTION

People share opinions about films in long, informal reviews. Sorting those reviews by hand is slow, and simple keyword searches miss context, negation and combinations of words. Sentiment analysis applies natural language processing to estimate whether a text expresses a positive or negative opinion. In this project, a trained LSTM network learns patterns from labeled IMDb reviews and exposes the result through a small local web application.

1.1 Project Scope and Planning

The project covers data loading, text cleaning, tokenization, sequence padding, neural-model training, held-out evaluation, artifact storage and single-review inference. The Streamlit interface accepts review text, displays a positive or negative result with a confidence score, and shows the model's test metrics. Sample-review buttons and a low-confidence warning support quick demonstrations. The work excludes neutral or multi-class labels, multilingual input, production hosting, a real-time review feed, and human moderation.

1.2 Need and Stakeholders

Stakeholder	Need
Students	A complete, reproducible NLP project for learning.
Instructors	A demonstrable workflow with actual held-out metrics.
Demo users	A simple way to test positive and negative review text.
Developers	Saved artifacts and clear instructions for running locally.

1.3 Contemporary Issues

- Text is ambiguous: sarcasm, mixed opinions and negation can reverse a keyword's apparent meaning.
- A benchmark score does not guarantee performance on current, informal or out-of-domain reviews.
- Confidence is the model's sigmoid-derived score, not a calibrated probability of correctness.
- Reviews can contain personal or offensive material; this demo does not store submissions or moderate content.

CHAPTER 1. INTRODUCTION (CONTINUED)

1.4 Task Identification

- Load the labeled IMDb train and test splits from the Keras dataset utility.
- Decode review token IDs, normalize text, and fit the tokenizer only on training text.
- Convert reviews to sequences and pad or truncate them to a fixed length of 200.
- Train the embedding-LSTM classifier with validation monitoring and early stopping.
- Measure predictions on the untouched 25,000-review test split and save the artifacts.
- Build and manually exercise a Streamlit interface using the saved model and tokenizer.

1.5 Technologies Used

Layer	Technology	Purpose
Language	Python	Training and application code
Deep learning	TensorFlow / Keras	Dataset loading, tokenization, LSTM model
Evaluation	scikit-learn	Accuracy, precision, recall, F1 and confusion matrix
Interface	Streamlit	Local browser-based review analysis
Storage	Keras .keras + pickle + JSON	Model, tokenizer and evaluation metrics

1.6 Project Timeline

Phase	Main work	Deliverable
Planning	Define binary task and report layout	Scope and project plan
Data preparation	Decode, clean, tokenize and pad reviews	Train/test arrays
Model development	Train and validate LSTM	Saved classifier
Evaluation	Predict on held-out data and calculate metrics	metrics.json
Application	Connect model to Streamlit and refine UI	Local web app
Documentation	Prepare figures, report and user instructions	Final report

CHAPTER 2.
LITERATURE REVIEW / BACKGROUND STUDY

2.1 Timeline of the Reported Problem

Early sentiment-classification research commonly represented text through word counts or hand-built features and trained conventional classifiers. The IMDb large movie-review dataset helped establish a substantial benchmark for binary polarity classification. Later neural approaches learned word representations and sequence patterns directly from labeled text. Recurrent networks, including LSTMs, were designed to retain useful information across longer input sequences, which makes them a reasonable educational model for reviews.

2.2 Existing Approaches

Lexicon-based systems score words using polarity dictionaries and are easy to explain, but may miss context and negation. Classical machine-learning systems combine bag-of-words or TF-IDF features with models such as logistic regression or support-vector machines. Neural methods learn representations with embeddings and can use word order; an LSTM is the sequence-based approach selected here. More recent transformer systems can capture richer context but need more computation and are outside this project's lightweight scope.

2.3 Literature Summary and Bibliometric Note

The references used here comprise foundational work on the IMDb sentiment dataset and LSTM networks, plus official documentation for the software APIs used in implementation. This is a focused background review, not a quantitative bibliometric survey: no database-wide publication counts or citation statistics were collected. The distinction avoids presenting unsupported publication-frequency claims.

2.4 Problem Definition

Given a movie review as a sequence of words, predict one of two labels: positive or negative. The main engineering requirements are consistent preprocessing between training and inference, a clearly separated held-out test set, saved artifacts that can be loaded by the app, and a user interface that communicates uncertainty.

2.5 Goals and Objectives

- Build a reproducible preprocessing and training pipeline for binary review polarity.
- Evaluate the learned model on a held-out IMDb split using class-aware metrics.
- Provide a simple local interface for user-entered reviews and transparent metrics.
- Document limitations and avoid presenting the output as a certain or universal judgment.

CHAPTER 3.
DESIGN FLOW / PROCESS

3.1 Evaluation and Selection of Features

The input feature is the review itself. It is represented as an ordered list of integer token IDs from the 10,000-word vocabulary. The model sees at most 200 tokens per review; sequences are padded with zeros or truncated at the end. The label is binary, with 0 for negative and 1 for positive. The app applies the same tokenizer and sequence length used in training.

3.2 Design Constraints

Constraint	Design choice / effect
CPU-friendly coursework demo	Small single-layer LSTM and five-epoch maximum
Fixed input shape	200-token sequences; longer reviews are truncated
Vocabulary limit	Words outside the top 10,000 map to OOV
Binary labels	No neutral class or emotion categories
Local demonstration	Streamlit process and model run on one machine

3.3 Design Flow

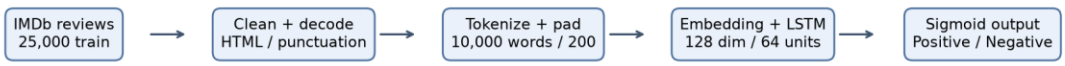


Fig. 3.1: Training and inference share the same review-normalization and sequence pipeline.

3.4 Design Selection

An embedding-LSTM network was selected because it combines learned word vectors with sequence order while remaining compact enough for local CPU execution. A single sigmoid unit directly represents the binary output score. Dropout reduces over-reliance on individual activations during training. Early stopping watches validation loss and restores the best weights seen during the run.

3.5 Implementation Plan / Methodology

The implementation is split between train.py, which builds and evaluates the model, and app.py, which loads the saved model and tokenizer for inference. Evaluation metrics are written to results/metrics.json. Keeping inference in the app on the same preprocessing path makes the model usable without repeating training.

CHAPTER 4.
RESULTS ANALYSIS AND VALIDATION

4.1 Implementation of the Solution

The training script loads 25,000 labeled training reviews and 25,000 test reviews. IMDb integer sequences are decoded with the dataset word index, cleaned, and passed to a Keras Tokenizer fitted on the training texts. Both splits are converted to padded arrays with a maximum length of 200. The model is trained with batch size 128 for up to five epochs, using 20% of the training portion for validation and early stopping with patience two.

The model has an Embedding layer (vocabulary 10,000, dimension 128), an LSTM layer with 64 units, Dropout (0.5), and a Dense sigmoid output. It uses Adam and binary cross-entropy. The 25,000 test reviews are used only after training to calculate final metrics. The resulting Keras model, tokenizer and JSON metrics are saved under model/ and results/.

Parameter	Value
Vocabulary / maximum length	10,000 / 200 tokens
Embedding / LSTM	128 dimensions / 64 units
Dropout	0.5
Optimizer / loss	Adam / binary cross-entropy
Batch size / maximum epochs	128 / 5
Validation / early stopping	20% / patience 2

Inference and User Interface

At runtime, the Streamlit application loads the saved artifacts, cleans the entered text, tokenizes and pads it to the same length, and runs model.predict. Scores at or above 0.5 are labeled positive; lower scores are negative. Confidence is the score for the predicted class. The app also presents test metrics and a confusion matrix, supports sample review buttons, and warns when confidence is below 60%.

Input-to-Output Architecture

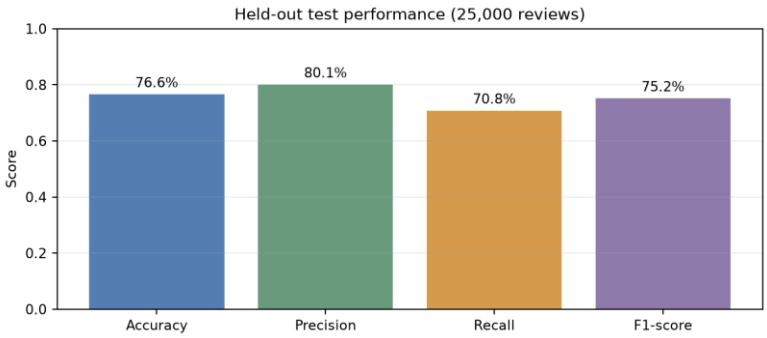
Browser text -> app.py cleaning -> saved tokenizer -> 200-token sequence -> Keras LSTM model -> label + score -> Streamlit result panel.

CHAPTER 4. RESULTS ANALYSIS AND VALIDATION (CONTINUED)

4.2 Results on the Held-Out Test Set

Metric	Value
Accuracy	0.7660
Precision	0.8010
Recall	0.7079
F1-score	0.7516

Table 4.1: Actual held-out test results, calculated on all 25,000 test reviews.



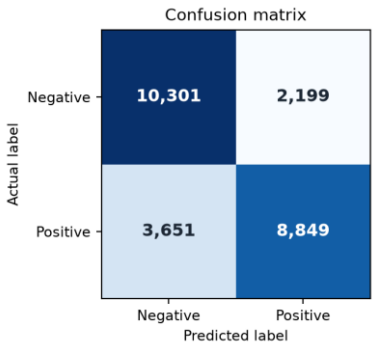


Fig. 4.1: Held-out metrics. Fig. 4.2: Confusion matrix (rows are actual labels; columns are predicted labels).

Actual / predicted	Negative	Positive
Negative	10,301 true negatives	2,199 false positives
Positive	3,651 false negatives	8,849 true positives

4.3 Validation and Discussion

The model correctly classified 19,150 of 25,000 reviews. Precision is higher than recall, indicating that some positive reviews are missed. The confusion matrix shows 3,651 false negatives and 2,199 false positives. Two example reviews were also run through the app; both were classified as expected, but their confidence values (51.4% and 58.4%) were close to the decision threshold and triggered the low-confidence notice. This confirms the warning is visible, while also showing that a correct class on a sample can still be uncertain.

The test score is evidence on this benchmark split only. It does not establish performance on reviews from other sources, languages or time periods. The training run uses one train/validation split and one final test evaluation; repeated splits and calibration would strengthen confidence in generalization.

4.4 APPLICATION SCREENSHOTS (WEB PAGES)

The running Streamlit interface accepts a movie review, returns a polarity label, and displays the raw score and confidence. The examples below are screenshots captured from the local application after applying the positive and negative sample buttons.

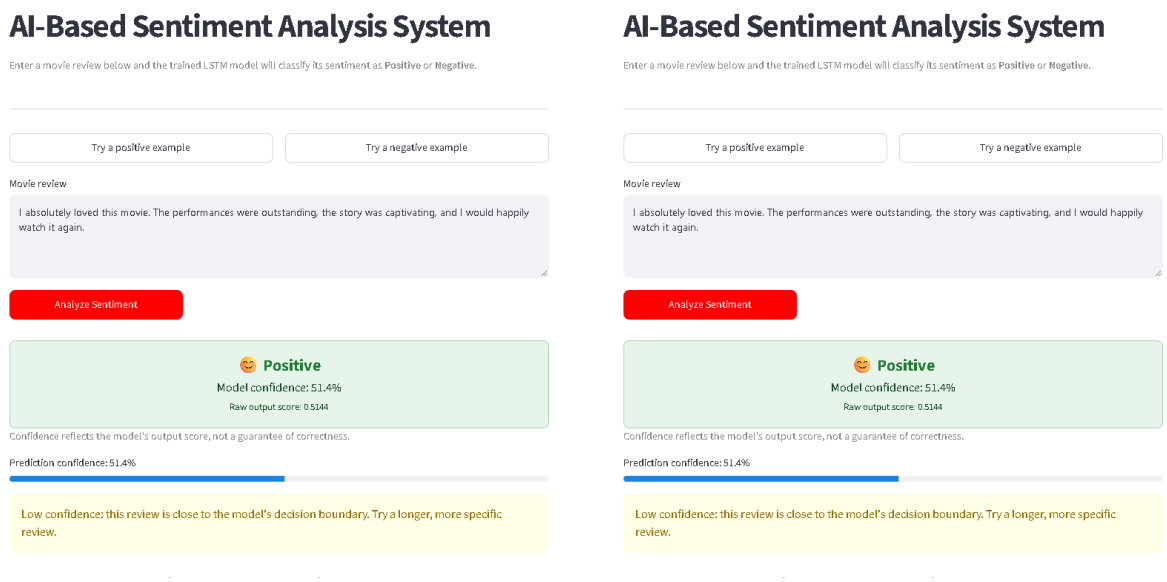


Fig. 4.3: Positive example (51.4% confidence; low-confidence warning). Fig. 4.4: Negative example (58.4% confidence; low-confidence warning).

Observed Interface Behavior

- The two sample buttons populate the review box without changing model artifacts.
- Analyze Sentiment runs the same cleaning, tokenizer and padding sequence used during training.
- The confidence bar and warning make borderline outputs visible to the user.
- Evaluation metrics remain available below the prediction result for context.

The examples are demonstration inputs, not a formal user study or a substitute for test-set evaluation.

CHAPTER 5.
CONCLUSION AND FUTURE WORK

5.1 Conclusion

This project implements an end-to-end binary movie-review sentiment classifier, from IMDb data loading and shared text preprocessing through LSTM training, test-set evaluation, saved artifacts and local Streamlit inference. On the 25,000-review held-out test set, it reached 76.60% accuracy and an F1-score of 75.16%. The app supports direct review entry, positive/negative examples, evaluation metrics and a low-confidence warning. The result demonstrates a working educational NLP pipeline, while the test score and low-confidence sample predictions show why individual outputs should be interpreted cautiously.

5.2 Future Scope

- Compare LSTM results with TF-IDF plus logistic regression and a compact transformer baseline.
- Use cross-validation, class-wise reporting, calibration and an external review dataset.
- Add batch CSV analysis, downloadable results and persistent user-controlled history.
- Explore multilingual support, neutral sentiment and explainability with careful validation.

APPLICATION CODE OF PROJECT

Core training configuration (train.py):

```
Embedding(10000, 128) -> LSTM(64) -> Dropout(0.5) -> Dense(1, sigmoid)
compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

Inference in app.py uses the saved Keras model and tokenizer, pads the token IDs to 200, and classifies the sigmoid output at a 0.5 threshold. The complete source files and trained artifacts accompany the project repository.

REFERENCES

1. Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., and Potts, C. (2011). Learning Word Vectors for Sentiment Analysis. ACL-HLT, 142-150. https://aclanthology.org/P11-1015/
2. Hochreiter, S., and Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735-1780. https://doi.org/10.1162/neco.1997.9.8.1735
3. TensorFlow. IMDb Movie Reviews dataset API. https://www.tensorflow.org/api_docs/python/tf/keras/datasets/imdb/load_data
4. Keras. Tokenizer and sequence preprocessing API. https://keras.io/api/preprocessing/text/
5. Keras. Recurrent layers: LSTM. https://keras.io/api/layers/recurrent_layers/lstm/
6. Streamlit. API reference: text_area, button and progress. https://docs.streamlit.io/develop/api-reference
7. scikit-learn. Classification metrics: accuracy, precision, recall, F1 and confusion matrix. https://scikit-learn.org/stable/modules/model_evaluation.html

APPENDIX: DESIGN CHECKLIST

Stage	Evidence	Status
Dataset	IMDb train/test split, binary labels	Complete
Preprocessing	Cleaning, tokenizer and fixed-length padding	Complete
Model	Embedding-LSTM classifier trained and saved	Complete
Evaluation	Held-out metrics and confusion matrix	Complete
Application	Streamlit inference and confidence feedback	Complete

USER MANUAL

1. Install Python 3.10+ and the packages listed in requirements.txt.
2. In the project folder, run: python train.py (only if the saved model files are absent).
3. Start the app with: streamlit run app.py; open the local URL shown in the terminal.
4. Enter a review or choose an example, then select Analyze Sentiment. Interpret low-confidence notices cautiously.

Troubleshooting: if the model is missing, run train.py; if packages are missing, install requirements.txt. The app requires local model/ files and internet only for the first IMDb dataset download during training.

GitHub repository: https://github.com/rahulparmar0578-svg/Ai-sentiment-analysis